Exercise - Placement / Timing

This exercise is still being updated for the 2026FS class. You can look at it but don't expect it to be in its final version until this message is displayed.

Learning Objectives

- Learn the steps to place your design on the floorplan you have established
- Be able to judge the quality of the placement, will this design manage to go through the subsequent routing stage?
- Understand static timing analysis results
- Experience how to fix timing issues

(A) Setup

About the style

We will use a number of different styles to identify different types of actions as shown below:

Student Task

Parts of the text that have a gray background, like the current paragraph, indicate steps required to complete the exercise.

Actions that require you to select a specific menu will be shown like the following: `menu → sub-menu → sub-sub-menu`

Whenever there is an option or a tab that can be found in the current view/menu we will use a `BUTTON` to indicate such an option.

Throughout the exercise you will be asked to enter certain commands using the command-line.

```
sh> command to be entered on the Linux command line
```

Note: *there are many reasons for using a command-line. Some functionality can not not be accessed through GUI commands, and in some cases, using the command-line will be much faster. Most importantly, things you enter on the command-line can be converted into a script and executed repeatedly.*

Preparation

Student Task 1

For ETH users:

- Setup the working directory for this exercise by calling our install script and navigating to the newly created directory. **Important**: make sure you install your exercises on the local hard disks rather than in the accounts home directories (which have a limited disk quota). For this sake, create a directory with your username on the local hard disk (e.g. `/scratch/vlsi2_XXfs26/`), and install the exercise files there. Check out the dedicated [Exercise setup]() page for more information.

```
sh > mkdir -p /scratch/$USER
sh > cd /scratch/$USER
sh > vlsi2 ex07
sh > cd ex07
```

Alternatively, you can directly call the install script for each exercise which you can find under `/home/vlsi2/ex07/install.sh`.

For non-ETH users:

Note: The exercises have been recently updated for the Spring semester 2026. It is possible that some exercise branches on Github require some additional time to be updated accordingly. Please reach out to a maintainer in case of issues.

- Retrieve the files from this exercise from Github:

```
sh > git clone https://github.com/pulp-platform/croc.git
sh > cd croc
sh > git checkout ex07
```

Student Task 2

- Initialize the exercise directory by running the Cockpit tool. This will create a predefined directory structure and copy the required technology for the IHP13SG2 process node files into the `technology` sub-directory. Please refer to the Exercise setup page for more details on the Cockpit tool. Make sure to run the `icdesign` command in the same directory where the `.cockpitrc` configuration file is located.

```
sh (ex07) > cd croc
sh (croc) > icdesign ihp13 -update all -nogui
```

(B) Understanding delays

Verifying functional correctness of a system using behavioral models does not necessarily mean that they also function correctly after fabrication.
Timing Analysis is an important verification step that needs to be done to ensure that the design meets the timing specifications. Therefore, in this exercise we begin by understanding delays of various elements in the design. Then we look into timing constraints from our understanding of delays. Then we go on to the Placement step of the backend design flow where the standard cells are placed after which we can already perform an initial evaluation of timing in our design.

Accurate timing analysis involves modelling,

- **Cell delays** - Which mainly depend on two factors (1) Input transition time (2) The load encountered by the cell.
- **Interconnect delays** - RC models to estimate the capacitance and resistances of the wires are used to model wire delays.

Below, is a sample path from input to output accumulating gate and wire delays. Here we have an example path starting at one of the input pins of the xnor gate to the output accumulating a delay of 0.5 ns, then accumulation wire delay of 0.1 ns due to resistance and capacitance of the metal layers, and a final delay of 0.3 ns through the mux.

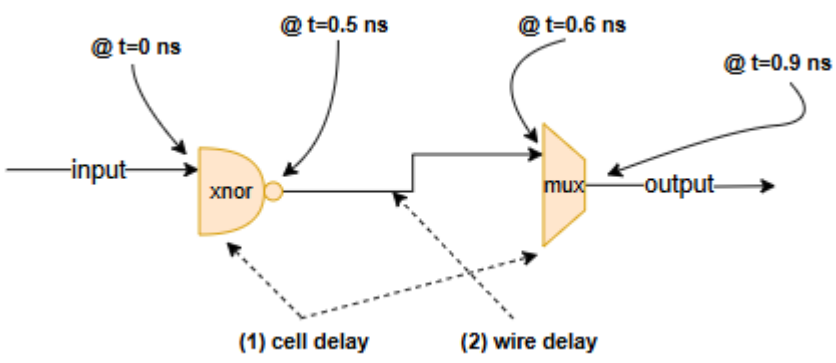

Figure 1: Cell and Interconnect delays.

Student Task 3

In this task, let's try to **understand standard cell delays**,

Open `technology/lib/sg13g2_stdcell_typ_1p20V_25C.lib`
("lib" stands for Liberty format with information of the functionality and electrical characteristics of the cell such as pin capacitances, delays, etc.)

1) From the first few lines what is the unit of delay in the lib file?
2) Search for `lu_table_template (TIMING_DELAY_7x7ds1)`.
Do you recognize the two variables that a cell delay depends on? What are they?

Some of the information can also be found on the datasheet pdf in `/usr/pack/ihp-sg13-kgf/open_ihp_sg13g2/IHP-Open-PDK-main_v2.0/sg13g2_stdcell_oa/doc/sg13g2_stdcell_typ_1p20V_25C.pdf`
1) Search for `NAND2x`. Verify the gate inputs, outputs, truth table
2) The `_1, _2, _4 ...` at the end of the cell name refers to the driving strength of the cell
Compare the Cell delays for a slew of 0.0186 ns and 0.001 pf load capacitance. Which cell has the lower delay?
Also compare which area footprint, pin capacitances, Leakage of the different driving strength cells.
3) One thing to note is the delay values calculated in the previous exercise is for a voltage of 1.2 V powering VDD of all standard cells and at room temperature 25C called typical corner (also indicated in the name of the lib file for example sg13g2_stdcell_typ_1p20V_25C.lib). You can also find a slow corner with timing information for a lesser voltage and a higher temperature and similarly a fast corner with higher voltages and a temperature of -40C. You will also see file with one more typical corner of 1.5 V, which we do not use as a library for the design since we aim our design at 1.2 V.

Student Task 4

Let's open a timing report and see the cell and interconnect delays in a design. Open `openroad/examples/timing_delay.rpt` which is a an example showing the timing for a path in the croc design.
The first few lines describe the path which we will see more later, but for now let us focus on understanding the delay of cells,
1) What is the delay of the mux `sg13g2_mux2_1` in units mentioned in the .lib file.
2) A mux has 3 inputs, 2 data inputs and one select signal. Through which of the pins of the mux does this path go through?
3) Is there another mux in this path? Is it of the same driving strength?
4) What does `net` between standard cells mean?
5) Is there a wire/net delay before the input arrives to the above `sg13g2_mux2_1` cell?

Now we now know where the delays come from and what kind of cells the tool has at its disposal to choose during synthesis and optimization steps.
Tool optimizations can be done to achieve various goals **timing, area, leakage power** etc. which are specified as constraints to the tool. Let's start and understand Timing constraints in detail.

(C) Timing Constraints

Two of the most important timing constraints are the setup and hold.

- **Setup** time of a Flip-Flop (FF) is the amount of time before the active clock edge at which the input should be ready before it can be sampled correctly. The setup timing constraints of all the FF within the design gives us the maximum operating frequency of the system. For a given clock constraint, **if the delay of a datapth is high**, then the output does not settle before the active clock edge. This means that the clock constraint is strict and needs to be relaxed so that output has enough time to settle before clock edge and meeting the setup constraint of the FF.
- **Hold** time of FF is the amount of time after the active clock edge for which data has to be held steadily before the next data arrives. Hold violations generally occur if the **data path is too short** such that a new signal propagated in the clock edge corrupts the sampling of the previous input in the same clock edge.

We depict the setup and hold timing diagram for 2 different sample paths. Let's look at a setup path shown in red going through xnor and mux. After the clock edge, we encounter the following delay of the FF, delay of xnor, delay of mux and wire delays. All these delays put together need to meet the setup timing constraint.
Let's look at another path in the design shown in blue. Here I show a datapath with only wire delay as an example of a path that could potentially cause hold violations. As you can see in this case of direct FF connection, we have the delay of the FF and the wire delay. This needs to be more than the hold time, such that this does not affect the sampling in the current clock edge.

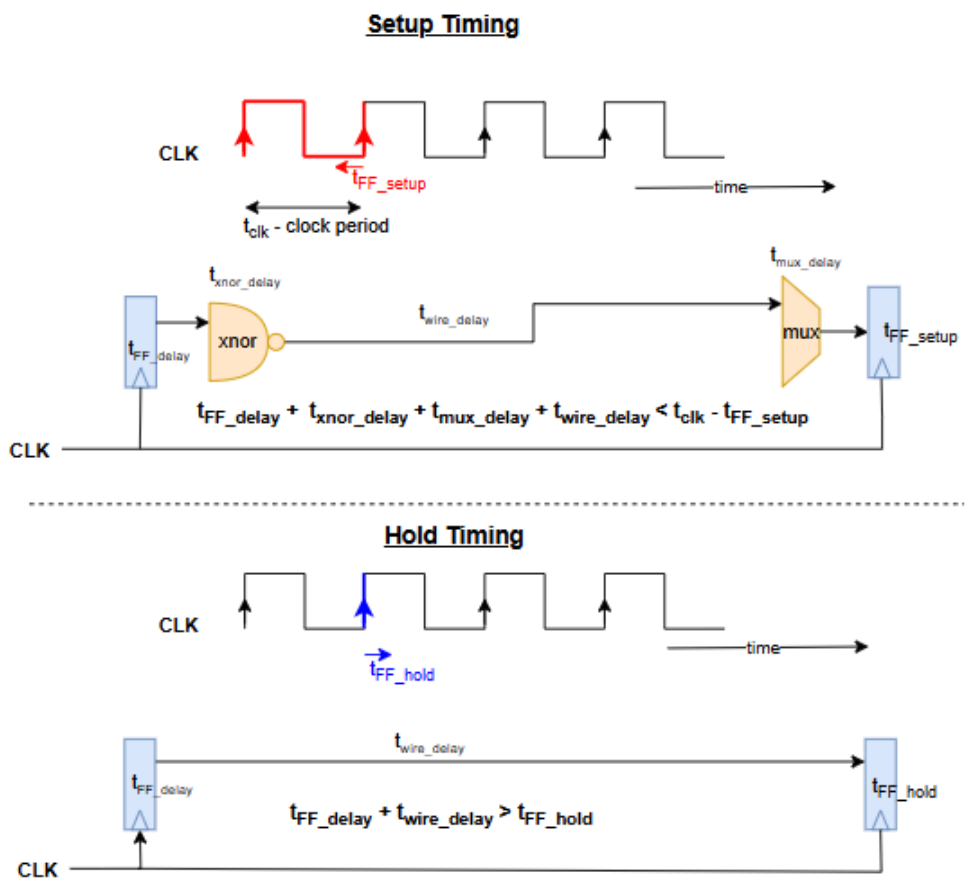

Figure 2: Depiction of setup and hold timing constraints.

A comparison of setup and hold timing violations is done below,
Setup violations:

- Violation of setup constraints arise due to the longer paths between two active clock edges. Setup violations limit the maximum clock frequency at which the system can be operated (The path that limits the frequency is the critical path of the system).
- Setup constraint is between consective clock edges (from the first clock edge when data is available at the FF output to the second clock edge where the data is sampled).
- Setup constraint is specified in the form of a clock period constraint to the tool. During the synthesis phase, the tool chooses the right cells from the library using higher driving strengts or insertion of buffers to reduce path delays to meet this constraint.

Hold violations:

- Violation of hold constraints arise because paths are short corrupting data sampled previously.
- Hold constraint is for the same clock edge.
- Hold constraint is not specified as a constraint to the tool. It is an intrinsic requirement for any design.

Student Task 5

What in your opinion is a more important timing requirement setup or hold timing? Or in other words what timing constraint violation would worry you the most?
Hint: Think what violations can be fixed after a chip is fabricated. Setup timing depends on the clock period.

Student Task 6

Let's do a simple **hand calculation of the setup and hold timing** constraints.
You can take the different wire and cell delays which are already mentioned in Figure 1,
For the FF assume the following parameters,
t_{FF_delay} = 0.1ns (Delay of the FF after clock edge), t_{FF_setup} = 0.2 ns (FF setup time), t_{FF_hold} = 0.2 ns (FF hold time)
Calculate if the setup and hold timing for the 2 paths is met for a clock period of 1 ns.

Student Task 7

Open the timing report `openroad/examples/timing_delay.rpt` again,
1) The startpoint and endpoint at the top identifies the path. This timing report is for a path from a FF to FF(reg2reg) denoted by `_reg` at the end.
Path Group tells which clock is feeding the launch and capture FF.
Path Type **max** correspond to setup and **min** corresponds to a hold path.
Now go to the end of the file,

2) Does the path meet the timing requirement of the design?
3) Can you recognize the clock period of `clk_sys` in from the timing report?
4) What is the **data arrival time**? (That is the time after the clock edge when the signal settles)
5) What is the **data required time**? (That is when the signal should have settled to meet the setup constraint of the FF)
6) Can you identify the setup time of the FF?
7) What should the clock period be to make sure the timing is not violated?

So one of the important timing constraint is the clock period of the system. Here the main clock of the croc system is the `clk_sys`. This is a sufficient enough constrain for all reg2reg (i.e. from a FF to FF) paths. However, additionally paths involving inputs and outputs for the chip needs to be considered and we additionally need to specify constraints for these paths. Input delay and output delay constraints for the inputs and outputs of the chip are explained below,

- **Input delay** is the time it takes for an input to settle before it enters the chip. In other words, to meet the setup timing, we have a delay of clock period reduced by the input delay coming from outside the chip.
- **Output delay** is the time taken for the signal from chip output to settle at the output sampling register. Similar to input delay, to meet the setup timing, we have a delay of clock period reduced by the output delay going outside the chip.

Usually the input and output delay are properties of the external system communicating with the chip. For example, they can be off-chip DRAMs, PCB wiring delays, etc. Sometimes, if you have a hierarchical design, the input/output delay also refers the amount time that needed outside your block (module) in top level.

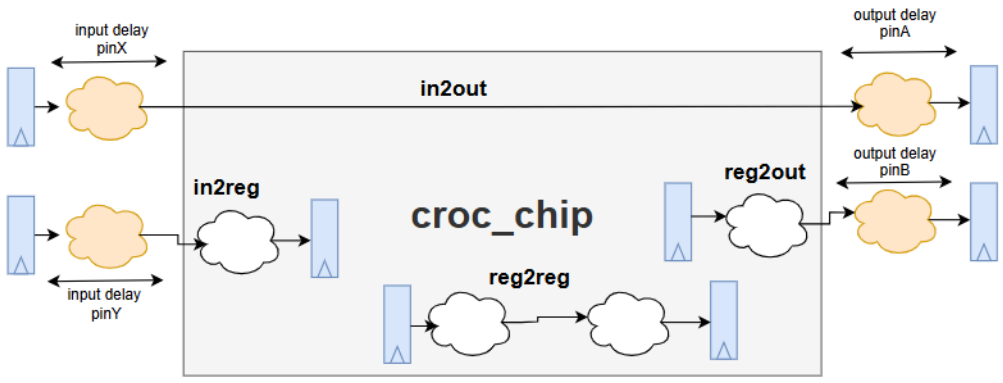

Figure 3: Depiction of different kinds of timing paths in the system with input and output delays.

(D) Placement

Having understood delays, how to look at timing constraints, let's move into the placement step of the physical implementation flow. From the synthesis exercise done in previous weeks, you generated a netlist and from the last floorplan exercise you learned how to place the macros, powergrid and the pads.
This exercise you will work with the placement of the standard cells in the netlist based on the constraints specified previously.

In the Placement step of the backend design, the core area of the chip (i.e the area excluding pads) is divided into rows where the standard cells from the netlist is to be placed. All standard cells within the library have the same height such that they can be placed side by side allowing the power pins to be connected on the same power rail of the power grid. The cells on above and below row are mirrored to share the power and ground supplies.

In the first step called **global placement**, the tool performs a rough standard cell placement according to the design data path and specified constraints. The tool can either focus on timing or routability (called Timing-driven or Routability-driven), but this coarse placement will not necessarily place the standard cell perfectly (they can still be overlapping with each other and not inside the rows). Then, the **detailed_placement** command performs detailed placement of instances to legal locations after global placement.

Let's open a design after the floorplan stage and perform the placement.

Student Task 8

Go into the `openroad` folder, start openroad and open design after the floorplan step

```
sh (openroad) > oseda -2025.01 openroad -gui
```

Setup the global variables for the project (project name, report and save directories)

```
OpenROAD > source scripts/init.tcl
```

Setup the technology libraries (standard cells, SRAM macro cells)

```
OpenROAD > source scripts/init_tech.tcl
```

Load the design database and constraints saved after floorplan stage

```
OpenROAD > read_db design/croc.pre_place.odb
OpenROAD > read_sdc design/croc.pre_place.sdc
```

Quickly explore the left panel in the GUI, What are the metal layers in use, the power grid, macros. Do you see standard cells?

Let's place the standard cells,

```
OpenROAD > set_thread_count 8
OpenROAD > global_placement
```

Also report cell usage and save it for later reference,

```
OpenROAD > report_cell_usage > reports/cellUsage_placement.rpt
```

Here an approximate placement of the standard cells is done. Observe the placement of the standard cells in the gui.
1) Is this a core limited or a pad limited design?
2) What is the placement utilization? This is also shown in the generated log during placement.

During placement we can see that the tool tries to place cells closer together to meet the timing constraints. However sometimes the cells can be densely placed due to limited placeable area. Due to dense packing of cells in an area, the routing will get complicated and requires more routing resources. In the later routing stage, this will bring detour nets or even shorts between the nets when the available routing tracks are no longer meet the desired nets in that area. The longer wires leading to larger delays which then affect timing. While timing is of concern, it is possible that the design is finished with many congested areas (hotspots) since the tool spends most of the efforts for timing and cares less about routing. Therefore it is very important to analyze the cell congestion after placement, and solve the congestion by either rearranging the macros (ie. improving the floorplan) or creating more space for those congested area, for example with paddings or placement blockages.

(E) Static Timing Analysis

So now we have done the placement, let's see if we can already meet the timing in the design.

Let’s generate timing reports after the placement step to understand the timing of the system as learned before, however 2 things to be noted
(1) The clock tree has not been generated yet, so we use an ideal clock that is all the registers are clocked at the same time.
(2) The real routing is not there, instead only a rough routing and estimation of parasitics are done in the placement stage.

These will be done in the following CTS (clock tree synthesis) and the Routing phase in the following exercises.

Student Task 9

Analyze the **timing report after the initial placement**,
On the right panel, open the **Timing Report** tab and run `Update` .

Let's analyze the setup timing which is in a tab named **Setup**,

1) Does the system meet the setup timing?

2) What is the worst negative slack?

3) Click on the critical path of the design (largest -ve slack) and trace the path shown on the design.

4) What is the Startpoint and Endpoint? Is this a reg2out, in2reg,.. path? Enable only the standard cells on the display control to trace the path better.

5) What is the total delay taken by the path from the Startpoint to Endpoint?

6) Right click on the critical path in the **Setup** tab and click **From Start to End**. A command appears on the console. This reports the timing for the path specific to the Startpoint and Endpoint.
Copy down the command `report_checks....` for later reference.

7) Rerun the report_checks command and save the path report to croc/reports/, to be used later.

```
OpenROAD > report_checks -from {....} -to {....} -fields {slew cap fanout input nets} > reports/criticalPath_placement.rpt
```

8) You can also save the design for future reference,

```
OpenROAD > save_checkpoint "${proj_name}.placement"
```

Student Task 10

Clearly this path does not meet the timing, Let's **analyze the timing path** further to see for issues,
1) Can you find the standard cell in the path that contributes most to the delay. What is this cell?
2) Why is the delay so high, based on the metrics reported for the path (Fanout/Slew/Load). Does this make sense from your understanding of a cell delay done previously?

Such a cell with enormous delay brings us the most important criteria (DRV) when it comes to fixing timing issue.

(F) Design Rule Violations (DRV)

Design Rule Violations (DRV) are metrics to evaluate to the quality of your chip. They are basically the set of factors based on which the design is characterized. Namely **max fanout, max capacitance and max transition** need to be within certain limits to achieve good timing for your design.
Max capacitance is the maximum capacitance value seen by output pin of a standard cell. Large output capacitance leads to higher standard cell delay. The liberty file defines the maximum capacitance that can be seen at the output pin of a cell.
Max transition/slew rate of the pin is the transition rise-to-fall/fall-to-rise time seen at the input pin. This is also defined in the liberty files for the standard cell inputs.
These violations need to be fixed to ensure the tool sees reasonable values for output load capacitance and input slew rates to accurately estimate cell delays. As seen before the lib file shows a lookup table based delay model. Extrapolating outside this range may not be accurate and hence need to be within limits. Another metric is the **Max fanout** which is the number of cells driven by the output cell. There is also a max fanout requirement specified in the liberty file.

We can also define these rules by ourselves to achieve a better timing result.

These violations need to be fixed to ensure that we can meet the timing of the design. The tool fixes these violations using the common below techniques,
1) Adding buffers - Adding buffers to split the fanout into multiple smaller fanouts. For example, a fanout of 32 can be fixed by adding 4 buffers. In this case, the driving cell has fanout of 4 and each inserted buffer has a fanout of 8. This constraint of max_fanout is also in the beginning of the .lib file
2) Upsizing the driving cell or downsizing the driven cell to reduce capacitance.

Student Task 11

Let's report the number of **DRV** violations first:

```
OpenROAD > puts "Vio: max_slew:[sta::max_slew_violation_count]  max_fanout:[sta::max_fanout_violation_count]  max_cap:
[sta::max_capacitance_violation_count]"
```

Then check how much they are violated:

```
OpenROAD > report_check_types
```

Output the detailed report for all the violations to get an idea what are they:

```
OpenROAD > report_check_types  -violators > reports/drv_placement.rpt
```

You will also find fanout violations (high fanout standard cell) in the critical path noted down previously, which also explains the poor timing at this point of the design after initial global placement.

Student Task 12

Before fixing the DRV violations, let's estimate the parasitics after placement to ensure parasitics are not ignored during DRV fixes,
To get a simple estimation in openroad, we can set it using set_wire_rc command:

```
OpenROAD > set_wire_rc -clock -layer Metal4
OpenROAD > set_wire_rc -signal -layer Metal4
OpenROAD > estimate_parasitics -placement
```

Run `Update` again, do you notice that the timing is worse now with added parasitic delays?

Now, **fixing of Design Rule violations** can be done in Openroad by the following command,

```
OpenROAD > repair_design -verbose
```

1) Look at the generated log and report how are the violations being fixed? Is this inline with your expectations as mentioned above?
2) Report the DRV violations again. Do you still see fanout violations?
The fanout violations have been fixed by the tool by addition of the buffers. This should significantly improve the timing.
You will see that the max slew and max capacitance has not been fixed. One reason could be that the tool finds that the fixing the fanout should already be enough to improve timing and ignore fixing the other violations.

Let's see if the timing has improved after the fixes,
3) In the GUI, `Update` the timing again to see the impact of the addition of buffers.
4) Does the design meet the setup timing after the DRV fixing step?
It can happen the design does not meet timing since the `repair_design` only majorly does DRV fixes.
5) What is the critical path now? Is it a reg2out, in2reg.. path?
6) If it is not a reg2reg path, open a reg2reg path clocked by **clk_sys** and verify that the path has no high fanout violations.
Also verify the addition of buffers with name `fanout*` that tool inserted to fix timing.
7) Report cell usage again to see how many buffers have been added during repairing.

```
OpenROAD > report_cell_usage > reports/cellUsage_fix.rpt
```

Compare with the previous `reports/cellUsage_placement.rpt`
8) The critical path now for reg2reg paths clocked by `clk_sys` need not exactly be the same as before. Let's report the critical path we had before we fixed the fanout violations to verify that this path indeed was fixed.
Run,

```
report_checks -from {....} -to {....} -fields {slew cap fanout input nets} > reports/criticalPath_fix.rpt
```

command saved before and verify that the path meets timing.
9) Also compare with the previous `reports/criticalPath_placement.rpt` or from the console and check if the high fanout violations are still there or not.

Student Task 13

Finally we can **legalize the placement**, that is to fix the placement of the cells to move the overlapping ones and align with the row

```
OpenROAD > detailed_placement
```

Below is the comparison view of the standard cells on rows before and after legalized placement.

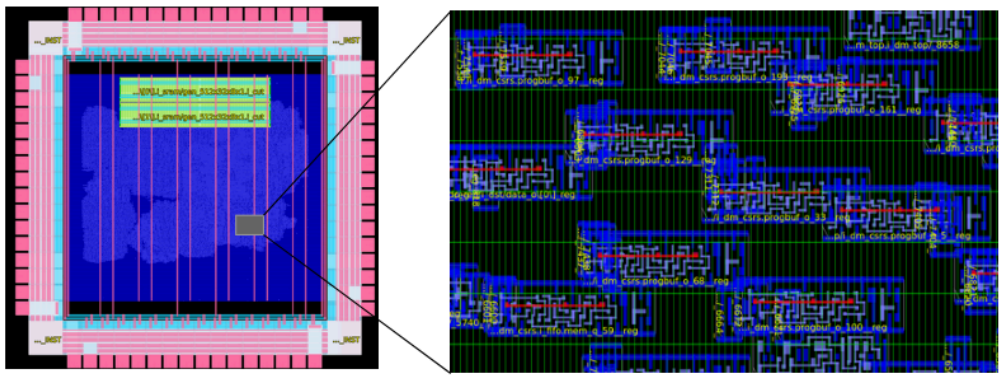

Figure 4: Approximate placement of standard cells during Global Placement.

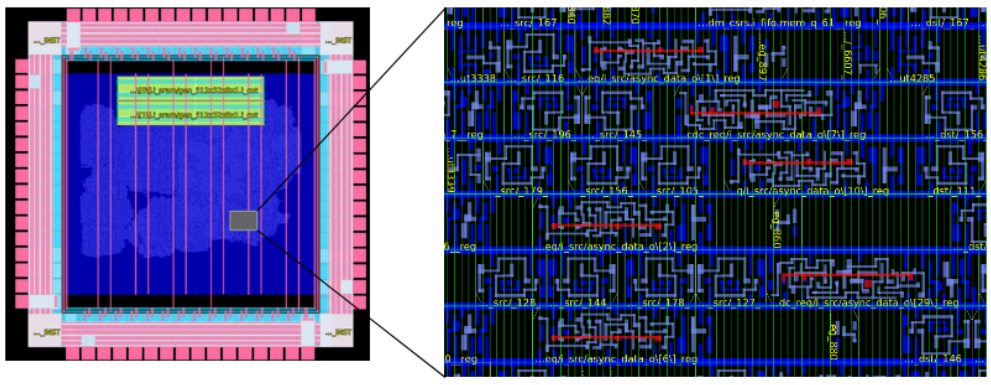

Figure 4: Legalized Placement of the standard cells.

Let's save this design checkpoint after fixes,

```
OpenROAD > save_checkpoint ${proj_name}.placement_fix
```

In the above tasks, we have looked at,

- Understanding cell delays
- Standard cell placement
- Static Timing Analysis
- Understanding DRV violations and impact on timing
- Fixing DRV violations

Note: We have ignored hold violations since placement stage is still too early to fix them. Because the clock tree is not yet synthesized, the real delay on the clock tree are not considered in our timing report. Hold violations can be fixed by the tool later using optimization techniques after clock tree synthesis and routing, which will be discussed in the following exercises.

The VLSI pages are part of the open source VLSI design course offered by the Integrated Systems Laboratory (https://iis.ee.ethz.ch) of ETH Zürich (https://www.ethz.ch), by Luca Benini (https://ee.ethz.ch/the-department/people-a-z/person-detail.luca-benini.html) and Frank K. Gürkaynak (https://iis-people.ee.ethz.ch/~kgf/). See full list of contributors. Unless otherwise noted content available under Creative Commons BY (https://creativecommons.org/licenses/by/4.0/).